

IIT JODHPUR

Feb'26 Minor Examination: EEL71020 Hardware Design for AI (ONLY for MTECH-AI Batch)

Guidelines (Total time: 120 minutes, Maximum Marks: 26):

- Please read the question paper very carefully. **NO clarification is required in any question.** In case of any doubt, assume whatever you wish to and state that in your answer.
- Usage of LLM/AI tool is NOT allowed. NOTES can be referred to. For Offline students, access to LAPTOP/MOBILE is NOT allowed.

1. Consider that your edtech company *PathshalaAI* plans to deploy a text-based classification model (Transformer-based, 110M parameters, similar to BERT-base) for sentiment analysis of the online student feedback.

The workload requirements are as follows:

- Batch size: 32
- Sequence length: 128 tokens
- Target throughput: $\geq 1,500$ inferences/sec
- Maximum allowed latency per batch: ≤ 25 ms
- Maximum power budget: 300 W
- Model size in FP16: 220 MB
- Estimated compute requirement per inference: 22 GFLOPs

The business procurement unit suggests to you the possible four choices out of which any one should be chosen for deployment of the above workload.

GPU-A	GPU-B	GPU-C	GPU-D
CUDA Cores: 2,048	CUDA Cores: 5,888	CUDA Cores: 10,752	CUDA Cores: 3,584
Tensor Cores: 64	Tensor Cores: 184	Tensor Cores: 336	Tensor Cores: 112
FP16 Throughput: 35 TFLOPS	FP16 Throughput: 125 TFLOPS	FP16 Throughput: 312 TFLOPS	FP16 Throughput: 70 TFLOPS
Memory: 8 GB GDDR6	Memory: 24 GB GDDR6	Memory: 40 GB GDDR6	Memory: 16 GB GDDR6
Memory Bandwidth: 256 GB/s	Memory Bandwidth: 600 GB/s	Memory Bandwidth: 1,550 GB/s	Memory Bandwidth: 384 GB/s
Power: 120 W	Power: 250 W	Power: 350 W	Power: 180 W
Cost: \$700	Cost: \$2,000	Cost: \$10,000	Cost: \$1,200

Given the workload requirements such as power efficiency, throughput etc., provide a recommendation of **the most suitable GPU configuration** with a detailed analysis? [4]

2. Suppose that a small deep-tech startup *LekhakaAI* incubated at IITJ hires you as a consultant for implementing CNN-based text recognition real-time implementation. The engineering team has implemented the core compute kernel for inference in a high-level language as shown below:

```
for (i = 0; i < 128; i++)
  for (j = 0; j < 128; j++)
    for (k = 0; k < 128; k++)
      C[i][j] += A[i][k] * B[k][j];
```

Consider that the total number of operations in the application/workload is slightly larger than 2.7 Million MACs per input and the target system constraints are mentioned below:

- Maximum latency per input: 3 ms
- Clock frequency: 300 MHz
- One MAC per cycle per compute unit
- Maximum available parallel MAC units: 400
- Memory is not the bottleneck.

How could the implementation of the above core compute kernel be carried out on any platform such that real-time performance is guaranteed? [4.5]

3. You are the engineering manager of a startup named as *FraudRoko* that deals with secure financial transactions. You decide to advise the custom silicon design team that is asked to implement a Random Forest (RF) classifier for tabular data analytics in your company's flagship financial fraud detection system. Consider the details of the RF model and the hardware system as shown below:

Random Forest Model Parameters	Hardware Architecture Assumptions
Number of trees: 100	All tree nodes are stored in off-chip global DRAM
Maximum depth per tree: 10	Feature vectors are also read from global DRAM
Internal nodes per tree (worst case full binary tree): =1023	On-chip SRAM size: 256 KB (so, it can cache part of random forest)
Each node stores: Feature index (4 bytes) Threshold value (4 bytes) Left & right child pointer (4 bytes each) Total per node = 16 bytes	Comparison operation cost: 1 floating-point compare
Feature vector size: 64 features	Tree traversal: One comparison per level One node access per level
Each feature: 4 bytes (FP32 representation)	Worst-case traversal depth = 10

Assume the workload and system characteristics as shown below:

- Batch size: 1 (single sample inference)
- Target throughput: 10,000 inferences/sec
- DRAM bandwidth: 12 GB/s
- Clock frequency: 500 MHz

You are aware that for a roofline model threshold, Peak compute capability = 50 GFLOPS and Memory bandwidth = 12 GB/s

(a) decide that whether the implementation is Memory-bound or Compute-bound? How? [1.5]

(b) Consider that instead of storing nodes in DRAM, if the first 5 levels of all trees are stored in on-chip SRAM, what happens to the system performance? How/Why this change could be a good decision? [2+2]

4. A startup is designing a custom accelerator for K-Means clustering to be deployed in an edge analytics device for industrial IoT anomaly detection. The accelerator will process streaming sensor data in real time through K-means processing of inputs. The algorithm specifications are as shown below:

- Number of clusters: $K = 7$
- Feature dimension: $D = 23$
- Maximum data points per batch: $N = 1,024$
- Distance metric: Squared Euclidean Distance
- Precision: Fixed-point Q8.8 format (Q8.8 is a 16-bit, fixed-point number format that uses 8 bits for the integer part and 8 bits for the fractional part. Therefore, 1 sign bit + 7 integer bits + 8 fractional bits)
- Maximum iterations allowed (during training): 12

The hardware specifications are shown as below:

- Clock frequency: 250 MHz
- Available DSP blocks: 96 (think DSP block as a specialized multiply-add hardware)
- Available BRAM: 512 KB (think BRAM as on-chip available memory)
- External memory bandwidth: 4 GB/s
- Only one global memory port
- No floating-point units allowed
- Accumulators are 32-bit fixed-point
- Each DSP can perform one 16×16 multiply per cycle
- Addition takes 1 cycle in LUT (Look Up Table) fabric

You have these 3 constraints that must be followed: (a) The distance-computation unit can process only 3 dimensions in parallel, (b) The centroid-update unit can process only 2 clusters in parallel, and (c) These two units cannot operate simultaneously (shared bus conflict which means that there is only one bus for the purpose of data transfer).

(a) Showing details, compute total operations (i.e., multiply and add) for worst-case 12 iterations? [2]

(b) Determine whether a fully sequential implementation can meet the deadline (≤ 5 ms). [3]

5. You are working in a consultancy firm as a senior partner. Considering the rapid advancement in AI technologies, a large number of your clients are companies dealing with AI implementations for real-life applications. As a part of your job, you decide to solve one such client's problem. This company *VisioSmart* is deploying a real-time image classification system using a CNN model on a GPU equipped with Tensor Cores, however, the company has a lots of fresher engineers and senior-level expertise is severely missing. The CNN model developed by AI developers of *VisioSmart* company, processes 224×224 RGB images and consists of the following layers as mentioned below:

Model Architecture	Target GPU Specifications
Layer 1: Convolution Input: $224 \times 224 \times 3$ Filters: 64 Kernel size: 7×7 Stride: 2 Output: $112 \times 112 \times 64$ Layer 2: Convolution Input: $112 \times 112 \times 64$ Filters: 128 Kernel size: 3×3 Stride: 1 Output: $112 \times 112 \times 128$ Layer 3: Fully Connected Input features: 2048 Output classes: 1000	Tensor Cores: 320 Each Tensor Core performs: 4×4 FP16 matrix multiply-accumulate per cycle 64 FP16 MACs per cycle GPU clock frequency: 1.2 GHz Peak FP16 Tensor Core throughput: 125 TFLOPS Global memory bandwidth: 900 GB/s Batch size: 32 Precision: FP16 compute with FP32 accumulation Target throughput: 500 images/sec

The engineers of *VisioSmart* seek your expert advise on the below points regarding performance enhancement in the above settings (MMA= Matrix-Multiply Accumulate).

- Would increasing batch size improve Tensor Core utilization? Why? [2]
- How should they tile matrices to align with Tensor Core MMA units? Explain. [3]
- Should convolution layers be fused with activation to reduce memory traffic? [2]